Achieving Blockchain-Secured Cryptographic Primitives from Signature-Based Witness Encryption

Lev Stambler
levstamb@umd.edu

Department of Computer Science, University of Maryland
Neon Tetra LLC

September 20, 2025

Abstract

Goyal and Goyal demonstrated that extractable witness encryption, when combined with smart-contract equipped proof-of-stake blockchains, can yield powerful cryptographic primitives such as one-time programs and pay-to-use programs. However, no standard model construction for extractable witness encryption is known, and instantiations from alternatives like indistinguishability obfuscation are highly inefficient.

This paper circumvents the need for extractable witness encryption by combining signature-based witness encryption (Döttling et al.) with witness encryption for KZG commitments (Fleischhacker et al.). Inspired by Goyal et al., we introduce $T+1$-Extractable Witness Encryption for Blockchains ($T+1$-eWEB), a novel primitive that encrypts a secret, making its decryption contingent upon the subsequent block's state. Leveraging $T+1$-eWEBs, we then build a conditional one-time memory, leading to a $T+1$ one-time program ($T+1$-OTP) also conditional on the next block's state.

1 Introduction

The celebrated Bitcoin paper provided a novel approach to solving a long standing set of questions in distributed systems, opening the floodgates to cryptocurrencies [1]. The key innovation of Bitcoin is an economic mechanism, rather than a purely computational one, that allows a set of nodes to agree on a single state of the system.

A line of works, started by Goyal and Goyal, asks a similar question for cryptographic primitives which are either impossible (such as obfuscation and one-time programs) or are challenging to achieve in a practical way [2]. Specifically, the work asks whether we can use blockchains to achieve cryptographic primitives which are *as secure* as the blockchain itself?

Ref. [2] uses a very advanced primitive, extractable witness encryption, to build a blockchain based one-time program. However, very little is known about extractable witness encryption.[1]

[1] To the author's knowledge, it is still not known if extractable witness encryption is possible in the standard model.

Thus, a line of works "emulate" extractable witness encryption using a distributed system armed with proactive secret sharing [3–7].

Unfortunately though, these new systems do not easily integrate into existing blockchains, and thus, to be considered secure (from an crypto-economic perspective), need to bootstrap a new blockchain or convince an existing blockchain to adopt a highly complex system.[2]

We are then left with a conundrum: how can we build primitives such as one-time programs that are as secure as the blockchain itself but also have the flexibility to be used in a wide variety of applications?

1.1 Our Contributions

Rather than trying to build a generic system for all proof of stake blockchains, we will focus on a blockchain which does three things:

1. Commit to its state via a KZG commitment [8].
2. Have the signing and verification of its committed state be done via a modified BLS signature [9].
3. Include its set of validators for the next block in the current block.

We then show how, via minor modifications to the BLS signature scheme, we can build a new primitive which we term **T + 1-eWEB** (T + 1-extractable witness encryption for blockchains). eWEBs were introduced by Goyel et al. [3] and allow for "encrypting" a secret to the blockchain and decrypting the secret if and only if a certain condition, relative to the blockchain state, is met. Our primitive is similar but only allows for decryption to occur if the condition is met at the block subsequent to the block in which the secret was encrypted. Importantly, our $T+1$-eWEBs are *as secure as the blockchain itself.* In order to decrypt the secret without the condition being met, an adversary must be able (to at least partially) forge (a signature on) the blockchain state! We present an illustration of $T+1$-eWEBs in Figure 1.

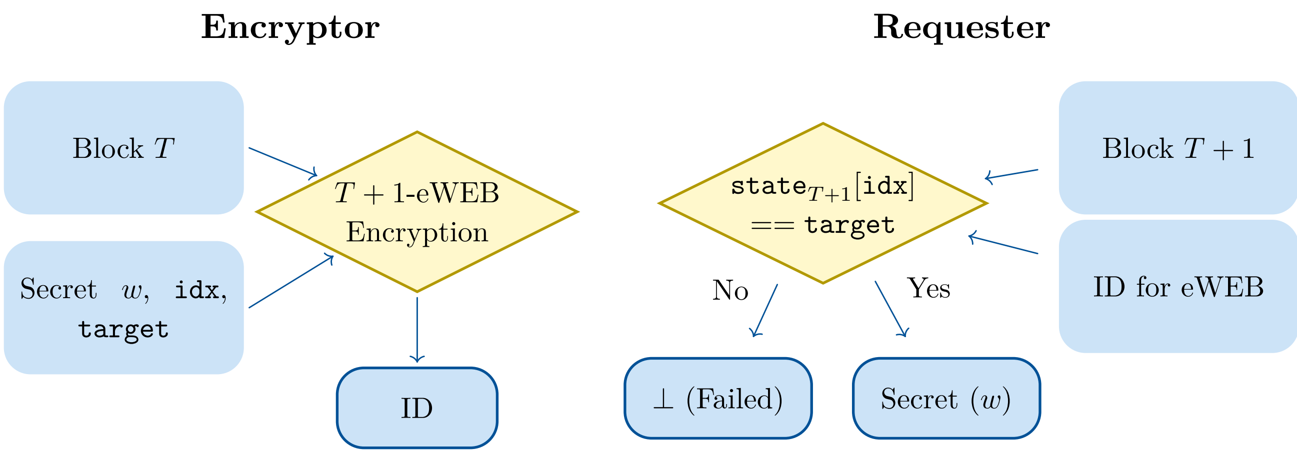

Figure 1: An illustration of the $T+1$-eWEB mechanism. The left panel shows the setup phase, where the mechanism outputs an identifier that will be used for retrieval. The right panel shows the reveal phase, where the requester checks the condition on the blockchain state and retrieves the secret if the condition is met.

[2]Most blockchains with a large market cap have a level of inherent "technical conservatism" as technical overhauls are quite difficult to achieve, have unintended consequences, and risk the entire system. It is therefore not surprising that leading blockchains have not adopted these systems.

Though $T+1$-eWEBs may seem limited, we show that they are quite powerful and can be used to build $T+1$ conditional one-time programs, which allow a program to be executed exactly once, contingent on a future blockchain state satisfying a condition. We note that this is a more general version of a $T+1$ pay-to-use program, which was introduced in Ref. [2]. The $T+1$ condition requires that the one-time program is evaluated prior to block $T+2$.

1.2 Technical Overview

Our construction is based on the following primitives: signature-based witness encryption (SWE), introduced in the McFly work [10] and extractable witness encryption for KZG commitments (WE-KZG) [11]. We will first briefly review these primitives and then outline our constructions.

SWE and WE-KZG

At a high level, SWE allows for encrypting a message m towards a signing committee V, threshold t, and tag T such that the encrypted ciphertext can only be decrypted given an aggregate signature of parties $U \subseteq V$ on tag T with $|U| \geq t$. In other words, a message can only be decrypted given a valid aggregate signature to a pre-specified tag. The construction presented in the McFly protocol [10] constructs SWE based on BLS-type signatures.

WE-KZG on the other hand, allows for encrypting a secret s under a KZG commitment $\texttt{comm}$ to state, $\texttt{state}$, such that s can only be recovered via a valid opening π to $\texttt{state}[\texttt{idx}] = \texttt{target}$ for $\texttt{idx}, \texttt{target}$ set by the encryptor. In other words, WE-KZG allows for encrypting a message to a specific opening to a commited vector.

High Level Idea and $T+1$-eWEBs

Key to this work, we change the BLS signature scheme outlined in the McFly protocol to sign a KZG commitment "bit-by-bit" rather than as a whole. So, if we have a commitment $\texttt{comm}$ to the blockchain state $\texttt{state}_T$, the signature does not sign $H(T \parallel \texttt{comm})$ for hash function H but rather signs $H(T \parallel i \parallel \texttt{comm}_i)$ for each bit i of the commitment. This modification allows us to use signature-based witness encryption alongside the fact that only *one* valid signature is produced per block to create a sort of "one-time" program for the *blockchain commitment*. Specifically, using garbled circuits, we can garble a circuit $C(\texttt{comm})$ which takes as input the KZG commitment $\texttt{comm}$. The garbling process then outputs wire labels $w_1^0, w_1^1, ..., w_{|\texttt{comm}|}^0, w_{|\texttt{comm}|}^1$. Then, we can encrypt the wire labels such that a user can only decrypt w_i^b if a valid blockchain signature, σ_i, is produced for message $H(T+1 \parallel i \parallel \texttt{comm}_i)$. Note that if the blockchain is secure, we never should have a signature be produced for both $H(T+1 \parallel i \parallel 0)$ and $H(T+1 \parallel i \parallel 1)$. Thus, after the blockchain signs the state commitment $\texttt{comm}$ for the $T+1$-th state, the garbled circuit can be executed on input $\texttt{comm}$ but should never be able to execute on $\texttt{comm}' \neq \texttt{comm}$.

At first glance, using SWE to encrypt a one-time program may seem to have a narrow use-case, but we now show how to use our second key primitive, extractable witness encryption for KZG commitments, which will allow us to make much more powerful primitives. Recall that extractable witness encryption for KZG commitments allows for encrypting a message towards a commitment to a state v, opening index i, and target t to get a ciphertext $\texttt{ct}_{\text{KZG}}$. Then, $\texttt{ct}_{\text{KZG}}$ can only be decrypted given a valid opening, π, of the committed state proving that $v[i] = t$. Armed with this primitive, we can now build build our $T+1$-eWEB for the following simple condition: for the next state, $\texttt{state}_{T+1}$ of a blockchain, we decrypt if $\texttt{state}_{T+1}[\texttt{idx}] = \texttt{target}$ for some predefined index $\texttt{idx}$ and target value $\texttt{target}$. Note that we cannot simply provide

$\mathtt{ct}_{\mathrm{KZG}}$ for the $T+1$-th state commitment of the blockchain as, at time T, we *do not know* what the $T+1$-th state will be! Instead, we use the previously outlined one-time program technique where the one-time program will "create" the ciphertext $\mathtt{ct}_{\mathrm{KZG}}$ given the state commitment $\mathtt{comm}_{T+1}$. We provide a high-level illustration of the setup in our construction in Figure 2 and the secret release in Figure 3.

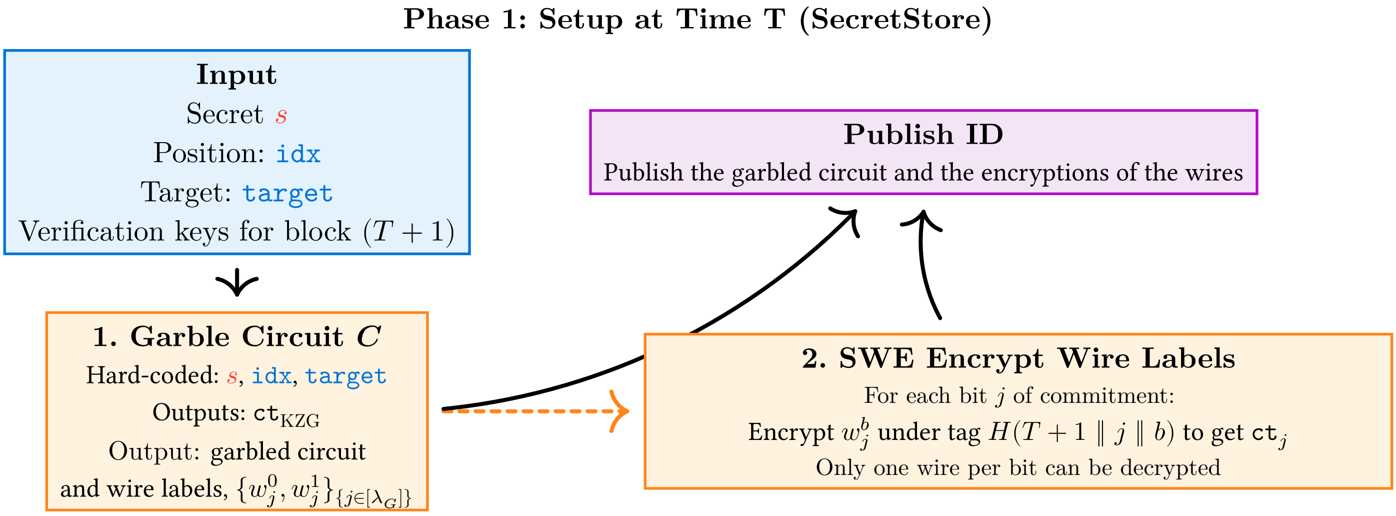

Figure 2: A visualization of the setup phase for our $T+1$-eWEB scheme.

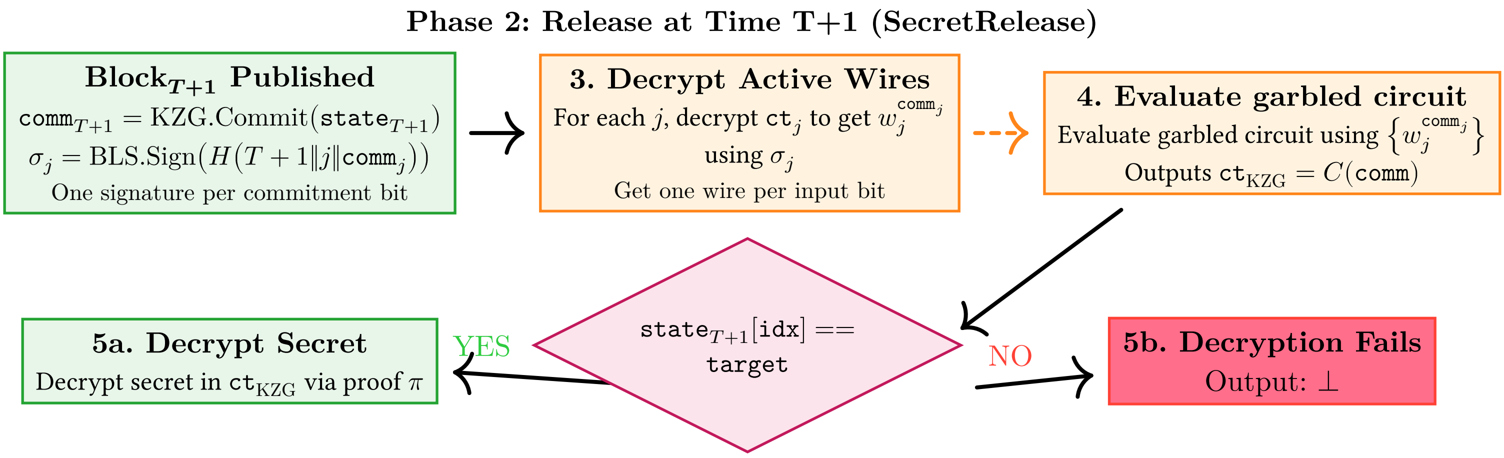

Figure 3: A visualization of the release phase for our $T+1$-eWEB scheme.

Constructing $T+1$ One-Time Programs from $T+1$-eWEBs

Recall that a $T+1$-eWEB allows us to store a secret such that the secret can only be retrieved if a specific data-slot on the blockchain has a target value. Given that a data-slot can only have *one* value for the $T+1$-th block[3], we can use our $T+1$-eWEBs two store two messages, where at most one of the messages can be decoded. In more detail, we would use our eWEB to encrypt m_0 requiring $\mathtt{state}_{T+1}[\mathtt{idx}] = 0$ to decrypt and encrypt m_1 requiring $\mathtt{state}_{T+1}[\mathtt{idx}] = 1$ to decrypt.

We now have built a *one-time memory* where only one-message can be decrypted. As shown by Goldwasser et al. [12], we can combine one-time memories with garbled circuits to then get one-time programs. And so, we can get $T+1$-one time programs, where we can evaluate the

[3] Otherwise, we would have *two different* (valid) blockchain states at block $T+1$, breaking the blockchain's guarantee of a unique and correct state.

program exactly once and only if the program's input is contained within the $T + 1$-th state of the blockchain.

Smart Contract Gating, Pay to Evaluate, and Variations on our Results
As noted by Goyal and Goyal [2], our use of the blockchain alongside obfuscation allows for a variety of new applications. Specifically, because program evaluation (for our OTPs and RAM obfuscation) is contingent on a blockchain state, we can have the setting of the program input be controlled by a *smart contract.* And so, we can require one of many possible conditions to be met before the program can be executed. For example, we can require that the program's input is set by a smart contract which requires a payment to be made as pointed out by Ref. [2]. This allows for a new class of "pay to evaluate" programs. We can also require that a certain number of (encrypted) transactions are posted to the blockchain prior to the program being executed, allowing for dark pools and other MEV-free financial applications.

1.3 Outline

In Section 2, we provide brief overviews of the cryptographic primitives on which we rely. In Section 3, we give an overview of our "blockchain" formalization, though not the same exact specification as the Ethereum blockchain, we aim to be as close as possible. Then, we introduce our $T + 1$-eWEB construction in Section 4 and use the construction to build one-time programs in Section 5. Finally, we conclude our work in Section 6.

2 Preliminaries

We now proceed to define the primitives which we will use in this work.

2.1 KZG Polynomial Commitments and BLS Signatures

Definition 2.1 *(KZG Polynomial Commitments [8])*: A KZG polynomial commitment scheme over field $\mathbb{F}_p$ consists of four algorithms $(\mathrm{Setup}, \mathrm{Commit}, \mathbf{Open}, \mathbf{Verify})$:

- $\mathtt{ck} \leftarrow \mathrm{Setup}(1^\lambda, 1^d)$: Generates a commitment key for polynomials of degree at most d.
- $\mathtt{comm} \leftarrow \mathrm{Commit}(\mathtt{ck}, f)$: Commits to a polynomial $f \in \mathbb{F}_p[X]$.
- $\pi \leftarrow \mathbf{Open}(\mathtt{ck}, f, \alpha, \beta)$: Generates an opening proving $f(\alpha) = \beta$.
- $b \leftarrow \mathbf{Verify}(\mathtt{ck}, \mathtt{comm}, \pi, \alpha, \beta)$: Verifies an opening.

Definition 2.2 *(BLS Signatures [9])*: The modified BLS signature scheme consists of algorithms $(\mathbf{KeyGen}, \mathrm{Sign}, \mathbf{Verify}, \mathrm{Agg}, \mathrm{AggVerify})$:

- $(\mathtt{vk}, \mathtt{sk}) \leftarrow \mathbf{KeyGen}(1^\lambda)$: Generates a verification key $\mathtt{vk} = g_2^x$ and signing key $\mathtt{sk} = x$.
- $\sigma \leftarrow \mathrm{Sign}(\mathtt{sk}, T)$: Signs a message T as $\sigma = H(T)^{\mathtt{sk}}$.
- $b \leftarrow \mathbf{Verify}(\mathtt{vk}, T, \sigma)$: Verifies a signature.
- $\sigma \leftarrow \mathrm{Agg}((\sigma_1, ..., \sigma_k), (\mathtt{vk}_1, ..., \mathtt{vk}_k))$: Aggregates signatures using Lagrange interpolation.
- $b \leftarrow \mathrm{AggVerify}(\sigma, (\mathtt{vk}_1, ..., \mathtt{vk}_k), (T_1, ..., T_k))$: Verifies an aggregate signature.

2.2 Signature-Based Witness Encryption

Definition 2.3 *(Signature-Based Witness Encryption [10])*: A t-out-of-n swe for an aggregate signature scheme $(\mathbf{KeyGen}, \mathrm{Sign}, \mathbf{Verify}, \mathrm{Agg}, \mathrm{AggVerify}, \mathbf{Prove}, \mathrm{Valid})$ is a tuple of two algorithms $(\mathbf{Enc}, \mathrm{Dec})$ where:

- $\texttt{ct} \leftarrow \mathbf{Enc}\Big(1^\lambda, V = (\texttt{vk}_1, ..., \texttt{vk}_n), \{T_i\}_{i\in[\ell]}, \{m_i\}_{i\in[\ell]}\Big)$: Encryption takes as input a set V of n verification keys of the underlying scheme Sig, a list of reference signing messages T_i and a list of messages m_i of arbitrary length $\ell \in \mathrm{poly}(\lambda)$. It outputs a ciphertext $\texttt{ct}$.
- $m \leftarrow \mathrm{Dec}\Big(\texttt{ct}, \{\sigma_i\}_{i\in[\ell]}, U, V\Big)$: Decryption takes as input a ciphertext $\texttt{ct}$, a list of aggregate signatures $\{\sigma_i\}_{i\in[\ell]}$ and two sets U, V of verification keys of the underlying scheme Sig. It outputs messages $\{m_i\}_{i\in[\ell]}$.

Definition 2.4 *(Robust Correctness, [10])*: A t-out-of-n SWE scheme $\mathrm{SWE} = (\mathbf{Enc}, \mathrm{Dec})$ for an aggregate signature scheme $\mathrm{Sig} = (\mathbf{KeyGen}, \mathrm{Sign}, \mathbf{Verify}, \mathrm{Agg}, \mathrm{AggVerify}, \mathbf{Prove}, \mathrm{Valid})$ is correct if for all $\lambda \in \mathbb{N}$ and $\ell = \mathrm{poly}(\lambda)$ there is no PPT adversary $\mathcal{A}$ with more than negligible probability of outputting an index $\mathrm{ind} \in [\ell]$, a set of keys $V = (\texttt{vk}_1, ..., \texttt{vk}_n)$, a subset $U \subset V$ with $|U| \geq t$, message lists $(m_i)_{i\in[\ell]}, (T_i)_{i\in[\ell]}$ and signatures $(\sigma_i)_{i\in[\ell]}$, such that

$$\mathrm{AggVerify}\Big(\sigma_{\mathrm{ind}}, U, (T_{\mathrm{ind}})_{i\in[\![U]\!]}\Big) = 1, \tag{1}$$

but

$$\mathrm{Dec}\bigg(\mathbf{Enc}\Big(1^\lambda, V, (T_i)_{i\in[\ell]}, (m_i)_{i\in[\ell]}\Big), (\sigma_i)_{i\in[\ell]}, U, V\bigg)_{\mathrm{ind}} \neq m_{\mathrm{ind}}. \tag{2}$$

Definition 2.5 *(Security)*: A t-out-of-n swe scheme $\mathrm{SWE} = (\mathbf{Enc}, \mathrm{Dec})$ for an aggregate signature scheme $\mathrm{Sig} = (\mathbf{KeyGen}, \mathrm{Sign}, \mathbf{Verify}, \mathrm{Agg}, \mathrm{AggVerify}, \mathbf{Prove}, \mathrm{Valid})$ is secure if for all $\lambda \in \mathbb{N}$, such that $t = \mathrm{poly}(\lambda)$, and all $\ell = \mathrm{poly}(\lambda)$, subsets $\mathrm{SC} \subset [\ell]$, there is no PPT adversary $\mathcal{A}$ that has more than negligible advantage in the experiment $\mathsf{Exp}_{\mathrm{SWE}}(\mathcal{A}, 1^\lambda)$. We define $\mathcal{A}$'s advantage by $\mathsf{Adv}^{\mathcal{A}}_{\mathrm{SWE}} = |\Pr[\mathsf{Exp}_{\mathrm{SWE}}(\mathcal{A}, 1^\lambda) = 1] - \frac{1}{2}|$.

Experiment $\mathsf{Exp}_{\mathrm{SWE}}(A, 1^\lambda)$

1. Let $H_{\mathcal{P}r}$ be a fresh hash function from a keyed family of hash functions, available to the experiment and $\mathcal{A}$.
2. The experiment generates $n - t + 1$ key pairs for $i \in \{t, ..., n\}$ as $(\texttt{vk}_i, \texttt{sk}_i) \leftarrow \mathrm{Sig}.\mathbf{KeyGen}(1^\lambda)$ and provides $\texttt{vk}_i$ as well as $\mathrm{Sig}.\mathbf{Prove}^{H_{\mathcal{P}r}}(\texttt{vk}_i, \texttt{sk}_i)$ for $i \in \{t, ..., n\}$ to $\mathcal{A}$.
3. $\mathcal{A}$ inputs $\mathrm{VC} = (\texttt{vk}_1, ..., \texttt{vk}_{t-1})$ and $(\pi_1, ..., \pi_{t-1})$. If for any $i \in [t-1]$, $\mathrm{Sig}.\mathrm{Valid}(\texttt{vk}_i, \pi_i) = 0$, we abort. Else, we define $V = (\texttt{vk}_1, ..., \texttt{vk}_n)$.
4. $\mathcal{A}$ gets to make signing queries for pairs (i, T). If $i < t$, the experiment aborts, else it returns $\mathrm{Sig}.\mathrm{Sign}(\texttt{sk}_i, T)$.
5. The adversary announces challenge messages m_i^0, m_i^1 for $i \in \mathrm{SC}$, a list of messages $(m_i)_{i\in[\ell]\setminus \mathrm{SC}}$ and a list of signing reference messages $(T_i)_{i\in[\ell]}$. If a signature for a T_i with $i \in \mathrm{SC}$ was previously queried, we abort.
6. The experiment flips a bit $b \leftarrow \{0, 1\}$, sets $m_i = m_i^b$ for $i \in \mathrm{SC}$ and sends $\mathbf{Enc}\Big(1^\lambda, V, (T_i)_{i\in[\ell]}, (m_i)_{i\in[\ell]}\Big)$ to $\mathcal{A}$.
7. $\mathcal{A}$ gets to make further signing queries for pairs (i, T). If $i \geq t$ and $T \neq T_i$ for all $i \in \mathrm{SC}$, the experiment returns $\mathrm{Sig}.\mathrm{Sign}(\texttt{sk}_i, T)$, else it aborts.
8. Finally, $\mathcal{A}$ outputs a guess b'.
9. If $b = b'$, the experiment outputs 1, else 0.

2.3 Extractable Witness KEM

Definition 2.6 *(Indexed Family of NP Relations)*: Let $I \subseteq \{0,1\}^*$ be a set. A set $\mathcal{F} = \{R_I\}_{I \in \mathcal{J}}$ is a family of NP relations with index set I if for all $I \in \mathcal{J}$, R_I is an NP relation. We call I the index of R_I and R_I the relation identified by I. We use $\mathcal{L}_I$ to refer to the corresponding NP language.

Definition 2.7 *(Witness Key Encapsulation Mechanism)*: A witness key encapsulation mechanism for a family of NP relations $\mathcal{F}$ and a keyspace $\mathcal{K}$ is a pair of PPT algorithms $\mathrm{WKEM} = (\mathsf{Encap}, \mathsf{Decap})$, defined as follows:

- $(\mathtt{ct}, k) \leftarrow \mathsf{Encap}(I, x)$: The encapsulation algorithm takes as input an index I identifying a relation $\mathcal{R}_I \in \mathcal{F}$ and a statement x and returns as output a ciphertext $\mathtt{ct}$ and a key $k \in K$.
- $k \leftarrow \mathsf{Decap}(I, w, \mathtt{ct})$: The deterministic decapsulation algorithm takes as input an index I identifying a relation $\mathcal{R}_I \in \mathcal{F}$, a witness $* w *$, and a ciphertext $\mathtt{ct}$ and returns a key $k \in \mathcal{K}$.

Definition 2.8 *(Correctness)*: A witness KEM $\mathrm{WKEM} = (\mathsf{Encap}, \mathsf{Decap})$ for a family of NP relations $\mathcal{F}$ is correct, if for any $\mathcal{R}_I \in \mathcal{F}$, any $\lambda \in \mathbb{N}$, any $(x, w) \in \mathcal{R}_I$, and any $(\mathtt{ct}, k) \leftarrow \mathsf{Encap}(I, x)$ it holds that $\mathsf{Decap}(I, w, \mathtt{ct}) = k$.

Definition 2.9 *(Extractability)*: A witness KEM $\mathrm{WKEM} = (\mathsf{Encap}, \mathsf{Decap})$ for a family of NP-relations $\mathcal{F}$ is extractable, if there exists a PPT algorithm Ext such that for any stateful PPT adversary $\mathcal{A}$ and any relation $\mathcal{R}_I \in \mathcal{F}$ such that

$$\Pr\left[\mathsf{Exp}^{\text{KEM-CPA}}_{\text{WKEM},\mathcal{A}}(1^\lambda, I) = 1\right] \geq \frac{1}{2} + \varepsilon(\lambda) \tag{3}$$

for some non-negligible function $\varepsilon(\lambda)$, it holds that

$$\Pr\left[(x, w) \in \mathcal{R}_I : x \leftarrow \mathcal{A}(1^\lambda, I); w \leftarrow \mathsf{Ext}^{\mathcal{A}(\cdot,\cdot)}(I, x)\right] \geq \delta(\lambda), \tag{4}$$

for some non-negligible function $\delta(\lambda)$. The latter probability is taken over the random coins of the adversary and the extractor and the experiment $\mathsf{Exp}^{\text{KEM-CPA}}_{\text{WKEM},\mathcal{A}}(1^\lambda)$ is defined as follows.

Experiment $\mathsf{Exp}^{\text{KEM-CPA}}_{\text{WKEM},\mathcal{A}}(1^\lambda, I)$

1. $x \leftarrow \mathcal{A}(1^\lambda, I)$ for relationship I
2. Sample $b \leftarrow \{0,1\}$
3. The experiment run the encapsulation algorithm to get test key k_0 $(\mathtt{ct}, k_0) \leftarrow \mathsf{Encap}(I, x)$
4. Sample a random key $k_1 \leftarrow \mathcal{K}$
5. $b' \leftarrow \mathcal{A}(\mathtt{ct}, k_b)$
6. Return 1 if $b = b'$ and 0 otherwise.

2.3.1 Witness KEM for KZG Commitments

We can view the work of Ref. [11] as a witness KEM for univariate polynomials where the relationship, I, is identified by a commitment $\mathrm{KZG}.\mathtt{comm}$ and a pair $((\alpha, \beta), \pi) \in \mathcal{R}_I$ if and only if $\mathrm{KZG}.\mathbf{Verify}(\mathtt{ck}, \mathrm{KZG}.\mathtt{comm}, \pi, \alpha, \beta) = 1$.

We formalize the above in the following definition:

Definition 2.10 *(Witness KEM for KZG Commitments)*: A witness KEM for KZG commitments is a tuple of algorithms $(\mathsf{Encap}, \mathsf{Decap})$, where:

- $(\mathtt{ct}, k) \leftarrow \mathsf{Encap}(1^\lambda, \mathtt{comm}, (\alpha, \beta))$: The encapsulation algorithm takes as input a KZG commitment, $\mathtt{comm}$, a point α in $\mathbb{F}_p$, and a value β in $\mathbb{F}_p$ and returns as output a ciphertext $\mathtt{ct}$.
- $k \leftarrow \mathsf{Decap}(\mathtt{comm}, w, \mathtt{ct})$: The decapsulation algorithm takes as input relationship $\mathtt{comm}$ identifying a relation $\mathcal{R}_{\mathtt{comm}}$, a witness w, and a ciphertext $\mathtt{ct}$ and returns a key k.

2.4 Garbled Circuits

Similar to Ref. [2], we use garbled circuits as defined by Ref. [13] though we modify the definition to allow for multi-bit outputs.

Definition 2.11 *(Garbled Circuit, [13,14])*: Let $\{\mathcal{C}_n\}_n$ be a family of circuits where each circuit in $\mathcal{C}_n$ has n bit inputs. A garbling scheme, $\mathtt{GC}$, for circuit family $\{\mathcal{C}_n\}_n$ consists of polynomial-time algorithms $(\mathtt{Garble}, \mathtt{Eval})$:

- $\mathtt{Garble}(1^\lambda, C \in \mathcal{C}_n)$: The algorithm takes a security parameter λ and a circuit C as input and outputs a garbled circuit G together with $2n$ wire keys $\{w_{i,b}\}_{i\in[n], b\in\{0,1\}}$
- $\mathtt{Eval}\left(G, \{w_i\}_{i\in[n]}\right)$: The evaluation algorithm takes as input a garbled circuit G and n wire keys $\{w_i\}_{i\in[n]}$ and outputs $y \in \{0,1\}^m$.

Definition 2.12 *(Garbling Correctness)*: A garbling scheme $\mathtt{GC}$ for circuit family $\{\mathcal{C}_n\}_n$ is said to be correct if for all λ, n, $x \in \{0,1\}^n$ and $C \in \mathcal{C}_n$,

$$\mathtt{Eval}\left(G, \left\{w_{i,x_i}\right\}(i \in [n])\right) = C(x), \tag{5}$$

where

$$\left(G, \left\{w_{i,b}\right\}_{i\in[n], b\in\{0,1\}}\right) \leftarrow \mathtt{Garble}(1^\lambda, C). \tag{6}$$

Definition 2.13 *(Selective Security)*: A garbling scheme $\mathtt{GC} = (\mathtt{Garble}, \mathtt{Eval})$ for a class of circuits $\mathcal{C} = \{\mathcal{C}_n\}_n$ is said to be a selectively secure garbling scheme if there exists a polynomial-time simulator Sim such that for all λ, n, $C \in \mathcal{C}_n$ and $x \in \{0,1\}^n$, the following holds:

$$\begin{aligned}\left\{\mathrm{Sim}(1^\lambda, 1^n, 1^{|C|}, C(x))\right\} \overset{c}{\approx} \Bigg\{ &\left(G, \left\{w_{i,x_i}\right\}_{i\in[n]}\right) \\ &: \left(G, \left\{w_{i,b}\right\}_{i\in[n], b\in\{0,1\}}\right) \leftarrow \mathtt{Garble}(1^\lambda, C)\Bigg\}\end{aligned} \tag{7}$$

We also require that if *no* wire keys are given, then the garbled circuit is indistinguishable from a random string of the same length.

Definition 2.14 *(Indistinguishability)*: A garbling scheme $\mathtt{GC} = (\mathtt{Garble}, \mathtt{Eval})$ for a class of circuits $\mathcal{C} = \{\mathcal{C}_n\}_n$ is said to be indistinguishable from random if for all λ, n, $C \in \mathcal{C}_n$, the following holds:

$$\left\{\mathrm{Sim}(1^\lambda, 1^n, 1^{|C|})\right\} \overset{c}{\approx} \left\{G : \left(G, \left\{w_{i,b}\right\}_{i\in[n], b\in\{0,1\}}\right) \leftarrow \mathtt{Garble}(1^\lambda, C)\right\}. \tag{8}$$

3 Blockchain Formalization

Rather than defining a fully fledged blockchain, we will define a more abstract and general notion of a "signature-based blockchain" (which we will denote $\mathcal{BC}$). All a signature-based blockchain needs to do is have a set of verification keys, a way to define "consensus" based on some threshold t of the signatures, and a way to define the state of the blockchain.

Definition 3.1 *(Blockchain Template)*: We define a blockchain as a tuple $\mathcal{BC} = (\mathbf{Verify}, \mathtt{state})$ where $\mathtt{state}$ is a mutable state of the blockchain and **Verify** is a *fixed* verification algorithm. Further, we have the following verification algorithm:

$$\mathbf{Verify}(\mathtt{state}') = 1 \tag{9}$$

if and only if $\mathtt{state}'$ is a valid state of the blockchain.

3.1 Signature Based Blockchain

We now narrow down the definition of a blockchain to a more specific notion of a "signature-based blockchain" off of which we will build. At a high level, a signature-based blockchain is a blockchain that has a set of verification keys, a way to verify signatures on the state of the blockchain, and a way to verify the set of verification keys.

Definition 3.2 *(Signature Based Blockchain)*: We define a signature-based blockchain at time T as a tuple $\mathcal{BC}_T = (\mathbf{Verify}_V, \mathtt{state}^*, t)$ where $\mathtt{state}^* = \mathtt{state}_1, ..., \mathtt{state}_{T-1}$ and $\mathbf{Verify}(\mathtt{state}^* \parallel \mathtt{state}_T) = 1$ if and only if for $\mathtt{vk}_1, ..., \mathtt{vk}_n \leftarrow V(\mathtt{state}_{T-1})$ and $\mathtt{state}_T = \left(\sigma_T, V_{T+1} \parallel \widetilde{\mathtt{state}}_T\right)$ is valid if and only if σ_T is a valid signature for $V_{T+1} \parallel \widetilde{\mathtt{state}}_T$ under at least t verification keys in $\mathtt{vk}_1, ..., \mathtt{vk}_t \subseteq V_T$. We also have that $\mathbf{Verify}(\mathtt{state}_1) = 1$ if and only if $\mathtt{state}_1$ is some the fixed initial state of the blockchain.

4 $T + 1$-Extractable Witness Encryption for Blockchain

We now present our modification on "extractable witness encryption" for blockchains (eWEBs) [3]. We will simplify the definition as, unlike Ref. [3], we will not need to worry about the underlying committees in the blockchain. Moreover, we restrict the eWEB to what we term a $T + 1$-eWEB: where decryption can only occur if the state of the *next* block[4] has some target state at position $\mathtt{idx}$ in the blockchain.

Definition 4.1 *($T + 1$-eWEB for Signature-Based Blockchain)*: For the T-th block and signature-based blockchain blockchain $\mathcal{BC}_T = (\mathbf{Verify}_V, \mathtt{state}^*, t)$ for $\mathtt{state}^* = (\mathtt{state}_1, ..., \mathtt{state}_T)$, let V_{T+1} be the verification algorithm for the $T + 1$-th block. Then, a $T + 1$-eWEB is a tuple of algorithms $(\mathtt{SecretStore}, \mathtt{SecretRelease})$ where:

- $\mathtt{SecretStore}(w, V_{T+1}, \mathtt{idx}, \mathtt{target})$: the sending party encrypts w which can be released if $\mathtt{state}_{T+1}[\mathtt{idx}] = \mathtt{target}$ and $\mathbf{Verify}_{T+1}(\mathtt{state}_{T+1}) = 1$. The secret then has an associated identifier, id
- $\mathtt{SecretRelease}(\mathrm{id}) \to w$ or $\perp$: a requester can request the secret w associated with id if $\mathtt{state}_{T+1}[\mathtt{idx}] = \mathtt{target}$ is true valid state $\mathtt{state}_{T+1}$. If $\mathtt{state}_{T+1}[\mathtt{idx}] = \mathtt{target}$ is not true, then the requester will get "bot" (or some other value) as the output.

We define the security of $T + 1$-eWEBs relative to the security of the underlying blockchain: i.e. only if we can produce two distinct valid states for the same block (and thus breaking the

[4] Here time does not literally refer to time on earth, but rather the block number.

blockchain), then we can break the security of the $T+1$-eWEB. Unfortunately, we are unable to give a "clean" and general definition of the security of $T+1$-eWEBs due to our scheme's construction in Section 4.1. Thus, we defer the formal definition to Section 4.1 where we will give a concrete instantiation of a time-based eWEB.

4.1 Instantiating Time-Based eWEBs

We now show how to instantiate time-based eWEBs via a series of circuits and smart-contracts. First though, we will need to define our underlying blockchain model.

4.1.1 Signature Based Blockchain Instantiation

Before we can fully construct our time-based eWEB, we need to make a few concrete "design decisions" about the underlying blockchain. We note that the blockchain itself can be constructed in a variety of ways, but for our purpose we need three main properties:

1. KZG polynomial commitments to the state at each time step
2. BLS signatures for the verification keys and signing of the root (though with a slight modification as in Ref. [10]).
3. And **bit-by-bit** signatures of the KZG root. Rather than directly signing the KZG commitment, `comm`, we will require a separate BLS signature for each **bit** of the KZG commitment when represented in binary.

We note that the first two properties are quite close to the existing Ethereum blockchain, but the third property is not.

We sketch out what the blockchain verification algorithm looks like in Blockchain 1.

Blockchain 1: Sketch of a Signature Based Blockchain. We do not specify a consensus algorithm

State $\texttt{state}^* = (\texttt{state}_1, ..., \texttt{state}_T)$ for the first T blocks where for blockchain data $\widetilde{\texttt{state}}_T$,

$$\texttt{state}_T = \left(\sigma_T, V_{T+1}, \widetilde{\texttt{state}}_T\right) \tag{10}$$

where

$$\sigma_j = \mathrm{Agg}\left(\left(\sigma_j^i\right)_{i \in U}, (\texttt{vk}_i)_{i \in U}\right) \tag{11}$$

and $\sigma_j^i = \mathrm{Sign}(\texttt{sk}_i, P_j)$ for

$$P_j = H\left(T \parallel j \parallel \underbrace{0...0}_{j-1}...b_j... \underbrace{0...0}_{\lambda_{\mathbb{G}}-j-1}\right) \tag{12}$$

and

$$b_j = \mathrm{Commit}\left(\texttt{ck}, V_{T+1} \parallel \widetilde{\texttt{state}}_T\right)[j], \text{ for } j \in [\lambda_{\mathbb{G}}] \tag{13}$$

$\textbf{Verify}(\texttt{state}^*) = 1$ if and only if:

For all $i \in 2, ..., T$

Let $V_i = (\texttt{vk}_1, ..., \texttt{vk}_n)$ be defined by $\texttt{state}_{i-1}$, $U_T \subseteq V_T$ and $|U_T| \geq t$, $\texttt{state}_T = \Big(\sigma_T = \left(\sigma_{T,1}, ...\sigma_{T,\lambda_{\mathbb{G}}}\right), V_{T+1}, \widetilde{\texttt{state}}_T\Big)$ is valid if and only if

$$\mathrm{AggVerify}\left(\sigma_{T,j}, (\texttt{vk}_i)_{i \in U_T}, P_j\right) = 1 \tag{14}$$

for all $j \in [\lambda_{\mathbb{G}}]$

And $\textbf{Verify}(\texttt{state}_1) = 1$ (i.e. the genesis block is publicly known and acknowledged)

As we can see in Blockchain 1, the verification algorithm is quite simple and *almost* standard except for the bit-by-bit signatures.

Now that our blockchain is defined, we define our security definition for the $T+1$-eWEBs. Specifically, the security will be defined relative to the security of the underlying blockchain where we will say that the blockchain is broken if the adversary can produce two *distinct* though valid signatures for the j-th bit of the KZG commitment. As in most proof of stake blockchains, we can still have a slashing condition which will allow us to "punish" any validators which produce two distinct signatures for the same bit of the KZG commitment.

Definition 4.2 *(Security of $T+1$-eWEB)*: Let signature-based $\mathcal{BC} = (\textbf{Verify}_V, \texttt{state}^*, t)$ be a blockchain with $V_{T+1} = (\texttt{vk}_1, ..., \texttt{vk}_n)$ being the verification keys for time $T+1$ and $\texttt{state}$ be the state of the blockchain. Let $\mathcal{BC}_0$ be the initial state of the blockchain and $\mathcal{BC}_T$ be the state of blockchain immediately prior to calling $\texttt{SecretStore}(s, \mathcal{BC}_T, \texttt{idx}, \texttt{target})$. We say that an $T+1$-eWEB is secure if for any BPP adversary $\mathcal{A}$ which can corrupt up to $t-1$ parties at any time, then the following holds: If for any $s, s', T, \texttt{idx}, \texttt{target}$,

$$\begin{aligned} \big|\Pr\big[b = 1 \mid b \leftarrow \mathcal{A}\big(1^\lambda, \mathcal{BC}, \texttt{SecretStore}(s, \mathcal{BC}_{\textbf{Init}}, T, \texttt{idx}, \texttt{target})\big)\big] - \\ \Pr\big[b = 1 \mid b \leftarrow \mathcal{A}\big(1^\lambda, \mathcal{BC}, \texttt{SecretStore}(s', \mathcal{BC}_{\textbf{Init}}, T, \texttt{idx}, \texttt{target})\big)\big]\big| > \mathrm{negl}(\lambda) \end{aligned} \tag{15}$$

then there exists a BPP extractor E such that for the check in eq. (14) with output $C_j \in \{0,1\}$ on $\texttt{state}^*$ and $\overline{C_j}$ on $\overline{\texttt{state}^*}$, and some $j \in [\lambda_{\mathbb{G}}]$:

$$
\begin{aligned}
\Pr\Big[&\mathtt{comm}_j \neq \overline{\mathtt{comm}}_j \quad \text{and} \quad C_j = 1, \overline{C_j} = 1 \\
&\text{or} \quad \mathbf{Verify}_V(\mathtt{state}^*) = 1 \quad \text{and} \quad \mathtt{state}_{T+1}[\mathtt{idx}] = \mathtt{target} \Big| \\
&\mathtt{state}^*, \overline{\mathtt{state}^*} \leftarrow E(1^\lambda, \mathcal{BC})\Big] \geq \mathrm{poly}^{-1}(\lambda).
\end{aligned} \tag{16}
$$

In words, our soundness says that if the adversary can break semantic security of the secret s without the condition being met, then the adversary can produce two distinct signatures for the same bit of the KZG commitment, breaking the blockchain[5].

4.1.2 eWEB Scheme

We are now ready to present our main technical contribution: our $T+1$-eWEB scheme, which will be the building block for the rest of the paper. Protocol 2 is our $T+1$-eWEB scheme which is based on the signature-based blockchain with verification procedure along the lines of Blockchain 1.

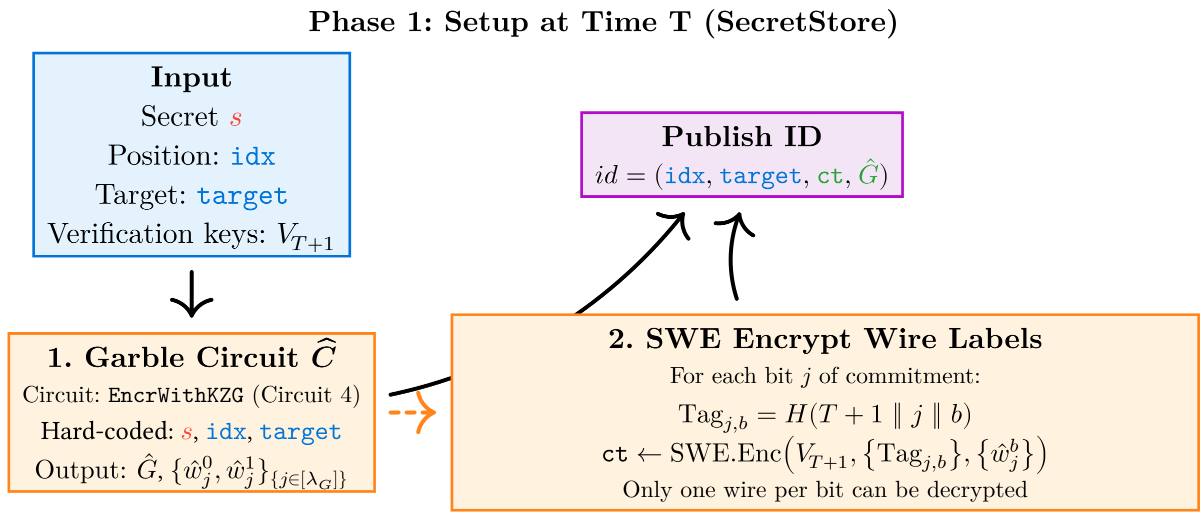

Figure 4: A visualization of the setup phase for our $T+1$-eWEB scheme.

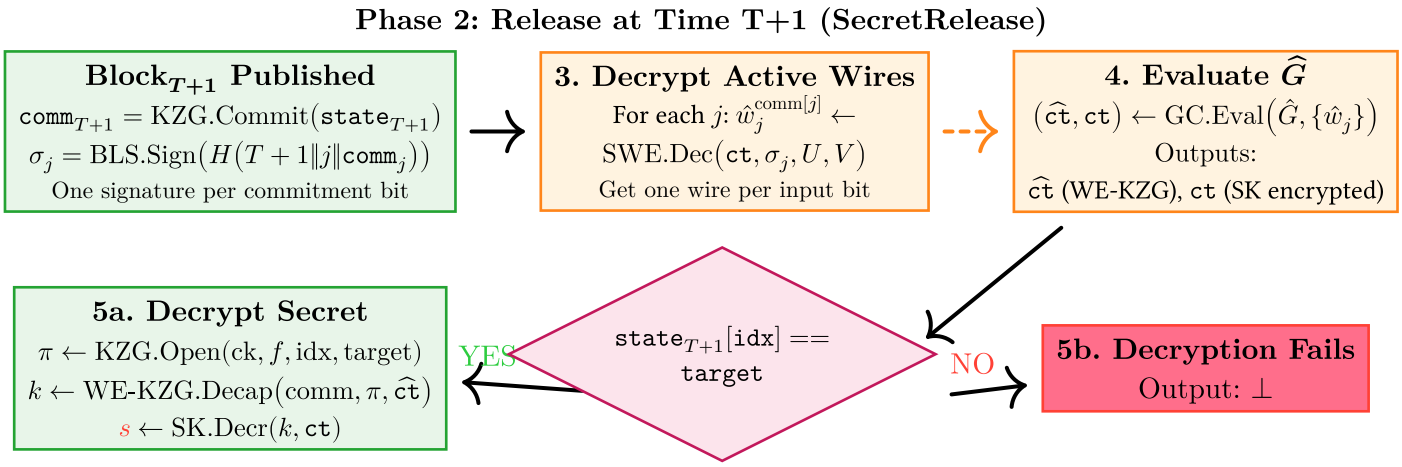

Figure 5: A visualization of the release phase for our $T+1$-eWEB scheme.

[5] Why can we not define the security in terms of an adversary producing two valid states? We use garbled circuits where security doesn't hold if the wire values for $0, 1$ for the same input bit are known to the adversary. As we shall see, if an adversary can produce two valid $\mathtt{comm}_j, \overline{\mathtt{comm}_j}$ with $\mathtt{comm}_j \neq \overline{\mathtt{comm}_j}$, then the adversary can decrypt two different wires for the same input bit, without necessarily having to come up with a fully valid blockchain state.

We provide a block-diagram visualization of the scheme in Figure 4 and Figure 5. Now, we will describe the formal details in Protocol 2 alongside a helper algorith, Algorithm 3, and a fixed circuit, Circuit 4.

Protocol 2: $T+1$-eWEB

$\texttt{SecretStore}(s, V_{T+1}, \texttt{idx}, \texttt{target})$ for message s and $V_{T+1} = (\texttt{vk}_1, ..., \texttt{vk}_n)$:

Garble the circuit $\hat{C}$ defined in Circuit 4 with hard-coded s, data position, $\texttt{idx}$, and target data, $\texttt{target}$. The labels are $\hat{w}_j^0, \hat{w}_j^1$ for $j \in [\lambda_{\mathbb{G}}]$ and the garbled circuit is $\hat{G}$.

Set

$$\texttt{ct} \leftarrow \text{SWE}.\textbf{Enc}\left(1^\lambda, V_{T+1}, \left\{T+1 \parallel j \parallel \underbrace{0...0}_{j-1}...b...\underbrace{0...0}_{\lambda_{\mathbb{G}}-j-1}\right\}_{b\in\{0,1\},j\in[\lambda_{\mathbb{G}}]}, \left\{\hat{w}_j^b\right\}_{b\in\{0,1\},j\in[\lambda_{\mathbb{G}}]}\right) \tag{17}$$

Publish $\text{id} = \left(\texttt{idx}, \texttt{target}, \texttt{ct}, \hat{G}\right)$

$\texttt{SecretRelease}\left(\text{id} = \left(\texttt{idx}, \texttt{target}, \texttt{ct}, \hat{G}\right)\right)$ for blockchain $\mathcal{BC}_{T+1}$:

For time $T+1$, let $U_{T+1} \subseteq V_{T+1}$ be the signing set and $V_{T+1} = (\texttt{vk}_1, ..., \texttt{vk}_n)$ be the verification keys.

If $|U_{T+1}| < t$

  Return $\perp$.

Call $\hat{w}_1^{\texttt{comm}_1}, ..., \hat{w}_{\lambda_{\mathbb{G}}}^{\texttt{comm}_{\lambda_{\mathbb{G}}}} = \texttt{DecrWires}(\sigma, V, U, \texttt{ct}, \texttt{comm})$ (Algorithm 3)

Let $\widehat{\texttt{ct}}, \texttt{ct} = \texttt{GC.Eval}\left(\hat{G}, \{\hat{w}_i^{\texttt{comm}_i}\}_{i\in[\lambda_{\mathbb{G}}]}\right)$ Use the wire labels $\hat{w}_i^{\texttt{comm}_i}$ to evaluate Circuit 4 to get $\widehat{\texttt{ct}}, \texttt{ct}$

Set $\pi = \text{KZG}.\textbf{Open}(\texttt{ck}, f, \texttt{idx}, \texttt{target})$

Set $k = \mathsf{Decap}(\texttt{comm}, \pi, \widehat{\texttt{ct}})$

Return $s = \texttt{sk.Decr}(k, \texttt{ct})$

Algorithm 3: $\texttt{DecrWires}(\sigma, V, U, \texttt{ct}, \texttt{comm})$, decrypt wire-labels for the first-stage garbled-circuit

Input: Signatures σ_i for $i \in [\lambda_{\mathbb{G}}]$, verification keys $V = (\texttt{vk}_1, ..., \texttt{vk}_n)$, signing set $U \subseteq [n]$, ciphertexts $\texttt{ct}$, and state root commitment $c = \texttt{comm}$.

For $i \in [\lambda_{\mathbb{G}}]$,

  Set $\hat{w}_i^{c_i} = \text{SWE.Dec}(\texttt{ct}, \sigma_i, U, V)$

Return wire labels $\hat{w}_i^{c_i}$ for $i \in \left[\lambda_{\text{garb}}\right]$.

Circuit 4: $\texttt{EncrWithKZG}(\texttt{comm})$, first-stage circuit to be garbled

Input: Commitment to KZG polynomial $\texttt{comm}$

Hard-Coded Values: Message s and fixed data-positions, $\texttt{idx}$, and target $\texttt{target}$

1 For $b \in \{0,1\}$, let $\widehat{\texttt{ct}}, k \leftarrow \mathsf{Encap}(1^\lambda, \texttt{comm}, (\texttt{idx}, \texttt{target}))$ as in Definition 2.10

2 Set $\texttt{ct} = \texttt{sk}.\texttt{Encr}(k, s)$

3 **Return** $\widehat{\texttt{ct}}, \texttt{ct}$

Theorem 4.1 (Correctness): Protocol 2 is correct (Definition 4.1) assuming that the signature-based witness encryption is correct, and the KZG base witness encryption is correct.

Proof: The proof follows from direct inspection of the protocol. If $\texttt{state}_{T+1}[\texttt{idx}] = \texttt{target}$, then the blockchain will produce a valid signature σ_j for the j-th bit of the KZG commitment, $\texttt{comm}$, for all $j \in [\lambda_{\mathbb{G}}]$. Then, we can decrypt the wire labels $\hat{w}_j^{\texttt{comm}_j}$ for all $j \in [\lambda_{\mathbb{G}}]$ using the signature-based witness encryption correctness (Definition 2.4). By the simulation soundness of the garbled circuits (Definition 2.13), we can evaluate the garbled circuit $\hat{G}$ on the wire labels $\hat{w}_j^{\texttt{comm}_j}$ to get $\widehat{\texttt{ct}}, \texttt{ct}$. Finally, we can use the KZG base witness encryption correctness (Definition 2.8) to decrypt $\widehat{\texttt{ct}}$ to get the message k and then decrypt $\texttt{ct}$ using the secret key k to get the message s. ■

Theorem 4.2 (Soundness): Protocol 2 is sound (Definition 4.2) assuming that the signature-based witness encryption is sound, and the KZG base witness encryption is sound.

We provide a full proof of the above theorem in Appendix A and sketch the proof here to provide intuition.

Proof sketch: The proof proceeds via contrapositive. Assume that there is no PPT extractor, E, which can produce two distinct signatures for the same bit of the KZG commitment for all $j \in [\lambda_{\mathbb{G}}]$ and that E cannot produce $\texttt{state}^*$ which is a valid blockchain state and meets the $T+1$-eWEB condition.

First, note that for all $j \in [\lambda_{\mathbb{G}}]$, either $\hat{w}_i^0$ or $\hat{w}_i^1$ is indistinguishable from random by the security of signature-based witness encryption (Definition 2.5) as the extractor cannot produce two valid signatures for the same bit of the KZG commitment. And so, we use the selective security of the garbled circuits (Definition 2.13) to replace the garbled circuit with the simulator which allows for evaluating C' at most once and learning nothing else about C'. And so, we can evaluate C' *only* on KZG commitment $\texttt{comm}_{\texttt{state}_{T+1}}$ as only the valid signatures for one KZG commitment are revealed. Next, note that the output of C', $\widehat{\texttt{ct}}, \texttt{ct}$ are indistinguishable from random by the security of the extractable witness KEM for KZG commitments (Definition 2.9) as $\texttt{state}_{T+1}[\texttt{idx}] \neq \texttt{target}$ and, by the binding property of the KZG commitment, no proof π can be produced for $\texttt{state}_{T+1}[\texttt{idx}] = \texttt{target}$ in polynomial time. ∎

5 Instantiating One-Time Programs

We now show why $T+1$-eWEBs, combined with a smart-contract system, are so powerful. We first start by instantiating $T+1$ one-time programs which allow for a single evaluation of a program by posting a message to the blockchain prior to the $T+1$-th block. We then show how $T+1$-OTPs can be bootstrapped to extremely powerful primitives such as RAM obfuscation. We also include "conditional gating" for the one-time programs which allows us to instantiate "pay-to-use" primitives and other powerful primitives. This section draws heavily from the ideas of Goyal and Goyal [2].

Definition 5.1 *($T+1$-conditional One-Time Program ($T+1$-OTP))*: A $T+1$-conditional one-time program, $T+1$-OTP, is a tuple $(\mathbf{Setup}, \mathbf{SetInput}, \mathbf{Eval})$ where:

- $\mathbf{Setup}(\mathcal{BC}_\ell, \texttt{Cond}, r)$ for $\ell \le T$ is a setup algorithm that takes as input a blockchain at time ℓ, circuit input size r, and a condition $\texttt{Cond}$ and outputs a setup identifier, $\mathrm{id}_{\mathbf{Setup}}$.
- $\mathbf{Init}(V_{T+1}, C, \mathrm{id}_{\mathbf{Setup}})$ is an initialization algorithm that outputs an identifier, $\mathrm{id}_{\mathbf{Init}}$
- $\mathbf{SetInput}(\mathcal{BC}_T, \mathrm{id}_{\mathbf{Setup}}, x)$ is an algorithm that takes as input a value x and sets the input of the OTP to x by updating the blockchain at time T.
- $\mathbf{Eval}(\mathcal{BC}_{T+1}, \mathrm{id}_{\mathbf{Init}})$ is an algorithm that takes as input the blockchain at time $T+1$ and attempts to output $C(x)$.

Definition 5.2 *(OTP Simulation-Based Soundness)*: We say that a OTP scheme is sound for circuit C relative and auxiliary information, $\texttt{aux}$, if:

- For every adversary, $\mathcal{A}$, there exists a simulator Sim such the following holds:

$$\begin{aligned} &\mathcal{A}\big(1^\lambda, 1^{|C|}, \mathcal{BC}_{T+1}, \mathrm{id}_{\mathbf{Setup}} \leftarrow \mathbf{Setup}(\mathcal{BC}_\ell, \texttt{Cond}), \texttt{Cond}, \mathbf{Init}\big(V_{T+1}, C, \mathrm{id}_{\mathbf{Setup}}\big)\big) \\ &\qquad \overset{c}{\approx} \mathsf{Sim}^{\texttt{Single}(C)}\big(1^\lambda, 1^{|C|}, \texttt{Cond}, \mathbf{Setup}(\mathcal{BC}_\ell, \texttt{Cond}), \mathcal{BC}_{T+1}, \texttt{aux}\big) \end{aligned} \tag{18}$$

where $\texttt{Single}(C)$ allows for only one evaluation of C if and only if $\texttt{Cond}(\texttt{state}_{T+1}) = 1$.

Notice that the simulator Sim also receives the setup output. Given that the setup only contains publicly known information, this is not a problem.

We leave a UC definition of $T+1$-conditional one-time programs for future work.

5.1 Constructing $T+1$-OTPs from $T+1$-eWEB

We now show how to instantiate $T+1$ one-time programs from $T+1$ eWEBs, assuming the existence of a smart contract system.

Smart-Contract 5: Smart Contract for One-Time Program

Hard-Coded values: State with position $\texttt{idx}_1, ..., \texttt{idx}_r$ for functions with r input bits and condition $\texttt{Cond}(\mathcal{BC})$.

Initialization:

Set $\mathcal{BC}[\texttt{idx}_i] = -1$ for all $i \in [r]$

SetInput(x) for $x \in \{0,1\}^r$:

If $\mathcal{BC}[\texttt{idx}_i] \neq -1$ for any i, then
  Return "Error: Input already set"
If $\texttt{Cond}(\mathcal{BC}) = 1$, then
  Set $\mathcal{BC}[\texttt{idx}_i] = x_i$ for all $i \in [r]$
If $\texttt{Cond}(\mathcal{BC}) = 0$, then
  Return "Error: Condition not met"

Protocol 6: $T + 1$-eWEB

Setup$(\mathcal{BC}_\ell, \texttt{Cond})$:

Post smart contract $\mathcal{SC}$ as defined in Smart-Contract 5 to the blockchain at time ℓ
Let $\text{id}_{\textbf{Setup}} = (\texttt{idx}_1, ..., \texttt{idx}_r, \texttt{addr})$ be the identifier of the smart contract and $\texttt{addr}$ be the address of the smart contract.
Return $\text{id}_{\textbf{Setup}}$

Init$\big(V_{T+1}, C, \text{id}_{\textbf{Setup}} = (\texttt{idx}_1, ..., \texttt{idx}_r, \texttt{addr})\big)$:

Garble circuit C with r input bits to wires $w_1^0, w_1^1 ..., w_r^0, w_r^1$ to get garbled circuit G.
For $i \in [r], b \in \{0,1\}$
  Let $\text{id}_i^b \leftarrow \texttt{SecretStore}\big(w_i^b, V_{T+1}, \texttt{idx}_i, b\big)$
Return $\text{id}_{\textbf{Init}} = \Big(G, \big\{\text{id}_i^b\big\}_{i\in[r], b\in\{0,1\}}\Big)$

SetInput$\big(\mathcal{BC}_T, \text{id}_{\textbf{Setup}}, x\big)$:

Call **SetInput** on $\mathcal{SC}$ at address $\texttt{addr}$ with the input x
If $\mathcal{SC}$ returns an error
  Return the same error

Eval$\Big(\mathcal{BC}_{T+1}, \text{id}_{\textbf{Init}} = \Big(G, \big\{\text{id}_i^b\big\}_{i\in[r], b\in\{0,1\}}\Big)\Big)$:

For $i \in [r]$
  Let $w_i^{x_i} \leftarrow \texttt{SecretRelease}\big(\text{id}_i^{x_i}\big)$
Call $y = \texttt{GC}.\textbf{Eval}\Big(G, \{w_i^{x_i}\}_{i\in[r]}\Big)$
Return y

Intuitively, we can see the above protocol as "enforcing" one-timeness by using a smart contract to ensure that the input is only set once.

Theorem 5.1 (One-Time Program Soundness from $T+1$-eWEB): Assuming the soundness of our $T+1$-eWEBs and that only one blockchain state can be produced per block, the protocol above is a sound $T+1$-conditional one-time program, $T+1\text{-}\mathsf{OTP}$, where soundness is defined as in Definition 5.2.

We give a proof sketch outline below and a more complete proof in Appendix B.

Proof sketch: The proof follows from a simple inspection of the smart contract alongside the security properties of the eWEB. Note that it, for any $i \in [r]$, $\mathtt{state}_{T+1}[\mathtt{idx}_r] \in \{-1, 0, 1\}$ and can only be set to *one* of these values by the soundness of the blockchain. Moreover, for all i, $\mathtt{state}_{T+1}[\mathtt{idx}_i] \neq -1$ if and only if $\mathtt{Cond}(\mathtt{state}_{T+1}) = 1$. And so, if $\mathtt{Cond}(\mathtt{state}_{T+1}) \neq 1$, then all the wires to the garbled circuit are indistinguishable from random by the security of the eWEB (Definition 4.2). Thus, if $\mathtt{Cond}(\mathtt{state}_{T+1}) \neq 1$, the simulator can simply simulate the garbled circuit (by the indistinguishability of garbled circuits, Definition 2.14.

If $\mathtt{Cond}(\mathtt{state}_{T+1}) = 1$ and $\mathtt{state}_{T+1}[\mathtt{idx}_i] \neq -1$, then an adversary can only learn the wires $w_i^{\mathtt{state}_{T+1}[\mathtt{idx}_i]}$ for $i \in [r]$ by the soundness of the $T+1$-eWEB. We then use the soundness of the garbled circuit to show that the simulator can simulate the garbled circuit and its wires while only learning $C(\mathtt{state}_{T+1}[\mathtt{idx}_1] \parallel \mathtt{state}_{T+1}[\mathtt{idx}_2] \parallel \ldots \parallel \mathtt{state}_{T+1}[\mathtt{idx}_r])$. ■

6 Conclusion

In this work, we show how to construct powerful cryptographic primitives where the security is based on the underlying security of a blockchain. Specifically, we construct a $T+1$-eWEB scheme and a conditional one-time program scheme for the $T+1$-th block's state root. Along the way, we give a simple yet formal description of a proof of stake blockchain model as well.

Though our construction is relatively simple, it is not without its drawbacks; follow up work is needed to improve the construction. First, our construction requires a modified BLS signature scheme with $\lambda_{\mathbb{G}}$ multiplicative blowup in the signature size. We are thus left with the following open questions:

Can we construct a scheme without $\lambda_{\mathbb{G}}$ multiplicative blowup in the signature size?

It seems as though we can use the aggregation of messages in BLS to reduce the size of the signature though this would require modifying the underlying SWE scheme.

Next, our construction only allows for a $T+1$-OTP where the program can only be run on the state root of the next block. Thus, we are left with the following open question:

Is there a simple way to extend our construction to allow for the program to be run at any time, not just on the next block's state root?

Finally, our construction is heavily based on pairing-based cryptography, a notably *quantum insecure primitive*. Unlike a blockchain, which uses cryptography for consensus, our primitives use crytpgraphy for privacy/secrecy as well. Thus, our work is vulnerable to "store-now-decrypt-later" attacks, where an adversary can store ciphertexts and decrypt them later once a functional quantum computer is available. So, we are left with the following open questions:

Can we construct a scheme that is post-quantum secure while simultaneously keeping its simplicity?

Acknowledgements

The author is grateful to the helpful discussions and feedback from Janmajaya Mall and Justin Drake. The author also acknowledges funding and support from Neon Tetra LLC and from the NSF Graduate Research Fellowship Program (NSF-GRFP).

Bibliography

[1] S. Nakamoto, Bitcoin: A Peer-to-Peer Electronic Cash System, (n.d.).

[2] R. Goyal and V. Goyal, Overcoming Cryptographic Impossibility Results Using Blockchains, (2017).

[3] V. Goyal, A. Kothapalli, E. Masserova, B. Parno, and Y. Song, Storing and Retrieving Secrets on a Blockchain, (2020).

[4] F. Benhamouda, C. Gentry, S. Gorbunov, S. Halevi, H. Krawczyk, C. Lin, T. Rabin, and L. Reyzin, Can a Public Blockchain Keep a Secret?, (2020).

[5] C. Gentry, S. Halevi, and V. Lyubashevsky, *Practical Non-Interactive Publicly Verifiable Secret Sharing with Thousands of Parties*, in *Annual International Conference on the Theory and Applications of Cryptographic Techniques* (2022), pp. 458–487.

[6] C. Gentry, S. Halevi, H. Krawczyk, B. Magri, J. B. Nielsen, T. Rabin, and S. Yakoubov, *Yoso: You Only Speak Once: Secure Mpc with Stateless Ephemeral Roles*, in *Annual International Cryptology Conference* (2021), pp. 64–93.

[7] V. Goyal, A. Kothapalli, E. Masserova, B. Parno, and Y. Song, *Storing and Retrieving Secrets on a Blockchain*, in *IACR International Conference on Public-Key Cryptography* (2022), pp. 252–282.

[8] A. Kate, G. M. Zaverucha, and I. Goldberg, Constant-Size Commitments to Polynomials and Their Applications, Advances in Cryptology - ASIACRYPT 2010 **6477**, 177 (2010).

[9] D. Boneh, B. Lynn, and H. Shacham, Short Signatures from the Weil Pairing, (n.d.).

[10] N. Döttling, L. Hanzlik, B. Magri, and S. Wohnig, McFly: Verifiable Encryption to the Future Made Practical, (2022).

[11] N. Fleischhacker, M. Hall-Andersen, and M. Simkin, Extractable Witness Encryption for KZG Commitments and Efficient Laconic OT, (2024).

[12] S. Goldwasser, Y. T. Kalai, and G. N. Rothblum, One-Time Programs, Advances in Cryptology – CRYPTO 2008 **5157**, 39 (2008).

[13] M. Bellare, V. T. Hoang, and P. Rogaway, *Foundations of Garbled Circuits*, in *Proceedings of the 2012 ACM Conference on Computer and Communications Security* (ACM, Raleigh North Carolina USA, 2012), pp. 784–796.

[14] A. C.-C. Yao, *How to Generate and Exchange Secrets*, in *27th Annual Symposium on Foundations of Computer Science (Sfcs 1986)* (1986), pp. 162–167.

A Proofs for T+1-eWEB

We now provide the missing proof for Section 4 and Theorem 4.2. We restate the theorem for convenience.

Theorem 1.1 (Soundness of Protocol 2): Protocol 2 is sound (Definition 4.2) assuming that the signature-based witness encryption is sound, and the KZG base witness encryption is sound.

Proof: The proof proceeds via contrapositive. Assume that there is no PPT extractor, E, which can produce two distinct signatures for the same bit of the KZG commitment for all $j \in [\lambda_{\mathbb{G}}]$ and that E cannot produce $\texttt{state}^*$ which is a valid blockchain state and meets the $T+1$-eWEB condition.

We now proceed to show that the adversary cannot distinguish between the two messages s and s' for any fixed $\texttt{idx}$ and $\texttt{target}$ given blockchain $\mathcal{BC}$ at block $F \geq T$ via a series of hybrid arguments.

- Hyb_0: The first hybrid is the real protocol where the adversary receives the output of $\texttt{SecretStore}$ and the blockchain $\mathcal{BC}_F$ for $F \geq T$
- Hyb_1: The second hybrid is the same except that if $F = T$, we replace all the wire labels in the encryption with random values, i.e. $\hat{w}_j^0, \hat{w}_j^1$ are replaced with random values for $j \in [\lambda_{\mathbb{G}}]$.
- Hyb_2: The same as Hyb_1 except that if $F > T$, then let σ be the signature for the $T+1$ block. Then let $\overline{\sigma}$ be the negation of σ ($\overline{\sigma_j} = 1 - \sigma_j$). Then, set $\hat{w}_j^{\overline{\sigma}_j} = \hat{r}_j$ where $\hat{r}_j$ is a random value for $j \in [\lambda_{\mathbb{G}}]$.
- Hyb_3: The same as Hyb_2 except that we replace the garbled circuit $\hat{G}$ with its simulator:
  - Let $\hat{G}$ be the garbled circuit for the first stage circuit Circuit 4 and the adversary's view is
  $$\hat{G}, \left\{\hat{w}_j^{x_j}\right\}_{j \in [\lambda_{\mathbb{G}}]}, \texttt{ct}, \mathcal{BC}, \texttt{idx}, \texttt{target} \tag{19}$$

  and gets replaced with simulator Sim_3 with view
  $$1^\lambda, 1^n, 1^{|C|}, C(x), \mathcal{BC}, \texttt{idx}, \texttt{target} \tag{20}$$
  where Sim_3 first simulates $\texttt{ct}$ and then uses the garbled circuit simulator. If $F = T$, then we set $C(x) = \bot$.
- Hyb_4: The same as Hyb_3 except that if $F > T$, we replace $\widehat{\texttt{ct}}, \texttt{ct} = C(x)$ with $\widehat{\texttt{ct}}, \texttt{ct}'$ where $\texttt{ct}'$ is an encryption of a random value.

Then, we can see in Hyb_4, as either $C(x) = \bot$ or $C(x)$ output $\texttt{ct}'$ which is an encryption of a random value, the adversary cannot distinguish between s and s' as we have removed s from the view of the adversary. ■

Lemma 1.1 *(*$\mathsf{Hyb}_0 \overset{c}{\approx} \mathsf{Hyb}_1$*)*: The first hybrid, Hyb_0, is computationally indistinguishable from the second hybrid, Hyb_1, we have a blockchain at time T and thus no signature is revealed for the $T+1$ block. And so, by the security of the signature-based witness encryption (Definition 2.5), the wire labels $\hat{w}_j^0, \hat{w}_j^1$ are indistinguishable from encryptions of random messages for all $j \in [\lambda_{\mathbb{G}}]$.

Lemma 1.2 *($\mathsf{Hyb}_1 \overset{c}{\approx} \mathsf{Hyb}_2$)*: Note that by our assumption, the extractor E cannot produce two distinct signatures for the same bit of the KZG commitment for all $j \in [\lambda_{\mathbb{G}}]$. And so, by the security of the signature-based witness encryption (Definition 2.5), *one* of the wire labels $\hat{w}_j^0, \hat{w}_j^1$ are indistinguishable from encryptions of random messages for all $j \in [\lambda_{\mathbb{G}}]$. Because the signature σ is revealed, then the indistinguishable wire labels correspond to $\overline{\sigma}$

Lemma 1.3 *($\mathsf{Hyb}_2 \overset{c}{\approx} \mathsf{Hyb}_3$)*: Define the circuit family for our garbled circuit $\mathcal{C}$ as

$$\mathcal{C} = \left\{ C(\mathtt{comm}) = \mathtt{EncrWithKZG}_{\mathtt{idx},\,\mathtt{target},s}(\mathtt{comm}) \mid s \in \mathcal{S} \right\} \tag{21}$$

where the subscript indicates the fixed values of the circuit. Then, by the previous two hybrids, at least one of the wire labels $\hat{w}_j^{\overline{\sigma}_j}$ is indistinguishable from random for all $j \in [\lambda_{\mathbb{G}}]$. And so, the simulator can choose a random value for $\hat{w}_j^{\overline{\sigma}_j}$ and encrypt it using the blockchain $\mathcal{BC}$. As the blockchain itself is independent of $\mathcal{C}$ as well, then we can use the selective security of the garbled circuits (Definition 2.13) to replace the garbled circuit $\hat{G}$ with its simulator.

Lemma 1.4 *($\mathsf{Hyb}_3 \overset{c}{\approx} \mathsf{Hyb}_4$)*: If $F = T$, then $C(x) = \perp$, and so s is removed from the view of the adversary. If $F > T$, then we have $\widehat{\mathtt{ct}}, \mathtt{ct} \leftarrow C(\mathtt{comm})$. Note that $\widehat{\mathtt{ct}}$ can only be decrypted to reveal k if an extractor can produce a valid proof π for $\mathtt{state}_{T+1}[\mathtt{idx}] = \mathtt{target}$. Moreover, note that we can only run $C(\mathtt{comm})$ once on a valid signature for block $T+1$. And so, $\mathtt{comm}$ must pass verification for $\mathtt{state}_{T+1}$ (i.e. $\mathbf{Verify}_{V(\mathtt{state}_{T+1})} = 1$) and, by our assumption, $\mathtt{state}_{T+1}[\mathtt{idx}] \neq \mathtt{target}$. Assume towards contradiction that the adversary can decrypt k and so an extractor exists, E', which can produce a valid proof π for $\mathtt{state}_{T+1}[\mathtt{idx}] = \mathtt{target}$ in polynomial time. And so, either the adversary has state $\mathtt{state}'$ which has commitment $\mathtt{comm}$ and $\mathtt{state}' \neq \mathtt{state}^*$, or the adversary has $\mathtt{state}^*$ which has commitment $\mathtt{comm}$ and $\mathtt{state}^*[\mathtt{idx}] \neq \mathtt{target}$. In the first case, the adversary breaks the binding property of the KZG commitment as it can produce two distinct signatures for the same bit of the KZG commitment. In the second case, the adversary can produce a valid proof π for $\mathtt{state}_{T+1}[\mathtt{idx}] = \mathtt{target}$ in polynomial time, breaking the soundness of the KZG commitment.

B Proofs for T+1-OTP

We now provide the missing details for the proof of Theorem 5.1. Recall the theorem statement:

Theorem 2.1 (One-Time Program Soundness from $T+1$-eWEB): Assuming the soundness of our $T+1$-eWEBs and that only one blockchain state can be produced per block, the protocol above is a sound $T+1$-conditional one-time program, $T+1$-OTP, where soundness is defined as in Definition 5.2.

We note that the following proof is a rather straightforward application of the soundness of the $T+1$-eWEB (Definition 4.2) and the security of the garbled circuits (Definition 2.13).

Proof: We proceed via a series of hybrid arguments.

- Hyb_0: the real protocol, here the view of the adversary is

$$\left\{ 1^\lambda, 1^{|C|}, \mathcal{BC}_{T+1}, \mathrm{id}_{\mathbf{Setup}} \leftarrow \mathbf{Setup}(\mathcal{BC}_\ell, \mathtt{Cond}), \mathtt{Cond}, \mathbf{Init}\left(V_{T+1}, C, \mathrm{id}_{\mathbf{Setup}}\right) \right\} \tag{22}$$

- Hyb_1: For all i, $\mathcal{BC}_{T+1}[\mathtt{idx}_i] = -1$, replace both w_i^0 and w_i^1 with random values.
- Hyb_2: If for all i, $\mathcal{BC}_{T+1}[\mathtt{idx}_i] \neq -1$, let $x_i = \mathcal{BC}_{T+1}[\mathtt{idx}_i]$. Replace $w_i^{1-x_i}$ with a random value.
- Hyb_3: If $\mathcal{BC}_{T+1}[\mathtt{idx}_i] \neq -1$ for all i, replace the garbled circuit with its simulator, giving away $C(x)$. We now have the simulated view

$$\left\{1^\lambda, 1^{|C|}, C(x), \mathtt{Cond}, \mathbf{Setup}(\mathcal{BC}_\ell, \mathtt{Cond}), \mathcal{BC}_{T+1}\right\}. \tag{23}$$

We can then see that in Hyb_4, if $\mathtt{Cond}(\mathtt{state}_{T+1}) = 0$, then, by the soundness of the blockchain, $\mathcal{BC}_{T+1}[\mathtt{idx}_i] = -1$ for all $i \in [r]$ and so the garbled circuit is indistinguishable from random by Definition 2.14. Otherwise, if $\mathtt{Cond}(\mathtt{state}_{T+1}) = 1$, by the security of the eWEB (Definition 4.2) and Definition 2.13, the simulator can only get one evaluation of C under input x. ∎

Lemma 2.1: $\mathsf{Hyb}_0 \overset{c}{\approx} \mathsf{Hyb}_1$

Proof: The first hybrid follows from the soundness of the eWEB (Definition 4.2) as we can replace the message with a random value if the blockchain's state at time $T+1$ does not match the target. ∎

Lemma 2.2: $\mathsf{Hyb}_1 \overset{c}{\approx} \mathsf{Hyb}_2$

Proof: The second hybrid follows from the soundness of the eWEB (Definition 4.2) as well: if $\mathcal{BC}_{T+1}[\mathtt{idx}_i] = x_i$ can only equal 0 or 1 and thus by the soundness of the eWEB, $w_i^{1-x_i}$ is removed from the view of the adversary as we can replace its encryption with an encryption of a random value. ∎

Lemma 2.3: $\mathsf{Hyb}_2 \overset{c}{\approx} \mathsf{Hyb}_3$

Proof: Note that the adversary's view can only contain at most one set of wires, $\{w_i^{x_i}\}_{\{i\in[r]\}}$. So, we can replace the garbled circuit with its simulator, which outputs $C(x)$, by the soundness of the garbled circuit (Definition 2.13). ∎